



MAPA – Material de Avaliação Prática da Aprendizagem

Acadêmico: Mauro Sérgio Santos da Silva Júnior	R.A.: 24150844-5
Curso: Bacharelado em Engenharia de Software	
Disciplina: Programação Orientada a Objetos	
Valor da atividade: 3,50	Prazo: 30/11/2025 23:59

Instruções para Realização da Atividade

1. Todos os campos acima deverão ser devidamente preenchidos;
2. Utilize deste formulário para a realização do MAPA;
3. Esta é uma atividade INDIVIDUAL. Caso identificado cópia de colegas, o trabalho de ambos sofrerá decréscimo de nota;
4. Utilizando este formulário, realize sua atividade, salve em seu computador, renomeie e envie em forma de anexo;
5. Formatação sugerida para esta atividade: documento Word, Fonte Arial ou Times New Roman tamanho 12, Espaçamento entre linhas 1,5, texto justificado;
6. Ao utilizar quaisquer materiais de pesquisa referencie conforme as normas da ABNT;
7. Critérios de avaliação: Atendimento ao Tema; Constituição dos argumentos e organização das Ideias.
8. Procure argumentar de forma clara e objetiva, de acordo com o conteúdo da disciplina.

Em caso de dúvidas, entre em contato com seu Professor Mediador.

Bons estudos!



1. Questão 1

- **Link do código completo no GitHub:**
<https://github.com/juniorsanttos1/Atividade-MAPA>

- **Texto Código Fonte:**

Classe ContaBancaria.java

```
public class ContaBancaria {

    // Atributos privados para encapsulamento
    private String titular;
    private int numeroConta;
    private double saldo;

    // Construtor da Classe
    public ContaBancaria(String nomeTitular, int numero) {
        this.titular = nomeTitular;
        this.numeroConta = numero;
        this.saldo = 0.0;
    }

    // Metodo para consultar saldo
    public double getSaldo() {
        return saldo;
    }

    // Metodo para consultar o nome do titular
    public String getTitular() {
        return titular;
    }

    // Metodo para consultar o numero da conta
    public int getNumeroConta() {
        return numeroConta;
    }
}
```



```
// Metodo para realizar deposito
public void depositar(double valor) {
    if (valor > 0) {
        this.saldo += valor;
        System.out.println("Depósito de R$ " + valor + "
                           realizado com sucesso!");
    } else {
        System.out.println("Valor de depósito invalido!");
    }
}

// Metodo para realizar saque
public void sacar(double valor) {
    // Verifica se o valor é positivo e se há saldo
    suficiente
    if (valor > 0 && valor <= this.saldo) {
        this.saldo -= valor;
        System.out.println("Saque de R$ " + valor + "
                           Realizado.");
    } else {
        System.out.println("Saldo insuficiente ou valor
                           invalido para saque.");
    }
}
}
```

Classe Sistema Bancário

```
public class SistemaBancario {
    public static void main(String[] args){
        // Inicia a conta bancaria com os dados iniciais
        ContaBancaria minhaConta = new ContaBancaria("Mauro",
        12345);
        System.out.println("Inicio das Operações");

        // Consultar o saldo inicial
        System.out.println("Saldo inicial: " +
        minhaConta.getSaldo());
    }
}
```



```
//Realiza um deposito
minhaConta.depositar(500.00);

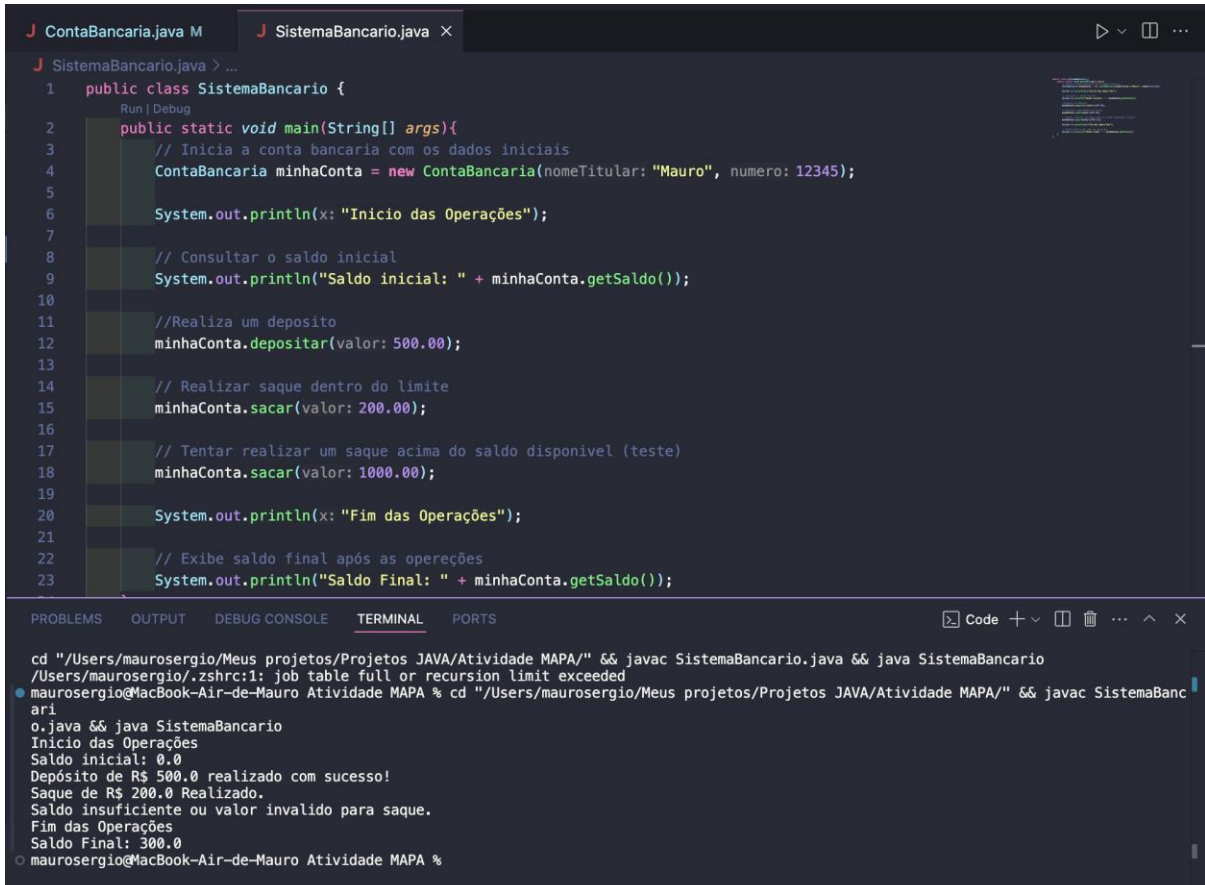
// Realizar saque dentro do limite
minhaConta.sacar(200.00);

// Tentar realizar um saque acima do saldo disponivel
// (teste)
minhaConta.sacar(1000.00);

System.out.println("Fim das Operações");

// Exibe saldo final após as opereções
System.out.println("Saldo Final: " +
    minhaConta.getSaldo());
}
}
```

- **Evidências de execução**



```
J SistemaBancario.java > ...
1 public class SistemaBancario {
2     public static void main(String[] args){
3         // Inicia a conta bancaria com os dados iniciais
4         ContaBancaria minhaConta = new ContaBancaria(nomeTitular: "Mauro", numero: 12345);
5
6         System.out.println(x: "Inicio das Operações");
7
8         // Consultar o saldo inicial
9         System.out.println("Saldo inicial: " + minhaConta.getSaldo());
10
11        //Realiza um deposito
12        minhaConta.depositar(valor: 500.00);
13
14        // Realizar saque dentro do limite
15        minhaConta.sacar(valor: 200.00);
16
17        // Tentar realizar um saque acima do saldo disponivel (teste)
18        minhaConta.sacar(valor: 1000.00);
19
20        System.out.println(x: "Fim das Operações");
21
22        // Exibe saldo final após as opereções
23        System.out.println("Saldo Final: " + minhaConta.getSaldo());

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
cd "/Users/maurosergio/Meus projetos/Projetos JAVA/Atividade MAPA/" && javac SistemaBancario.java && java SistemaBancario
/Users/maurosergio/.zshrc:1: job table full or recursion limit exceeded
maurosergio@MacBook-Air-de-Mauro Atividade MAPA % cd "/Users/maurosergio/Meus projetos/Projetos JAVA/Atividade MAPA/" && javac SistemaBanc
ari
o.java && java SistemaBancario
Inicio das Operações
Saldo inicial: 0.0
Depósito de R$ 500.0 realizado com sucesso!
Saque de R$ 200.0 Realizado.
Saldo insuficiente ou valor invalido para saque.
Fim das Operações
Saldo Final: 300.0
maurosergio@MacBook-Air-de-Mauro Atividade MAPA %
```



2. Questão 2

O encapsulamento é fundamental para a segurança e manutenção do código, pois protege a integridade dos dados ao restringir o acesso direto aos atributos da classe, utilizando modificadores como `private`. Isso obriga que qualquer alteração passe por métodos públicos que contêm regras de validação, impedindo estados inconsistentes, como um saldo negativo. Além disso, essa prática centraliza a lógica de negócios, facilitando a manutenção futura, já que alterações nas regras internas da classe não afetam o restante do sistema que a utiliza.